



Università
Ca'Foscari
Venezia

Generative Models

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

Andrea Torsello

Dip. Sc. Ambientali, Informatica, Statistica
Università Ca'Foscari Venezia

Classification Through Boundary Estimation

So far, we've mainly been talking about learning algorithms that model $p(y|x; \theta)$, the conditional distribution of output y given the input x .

- ▶ Linear classifiers model the decision on $p(y|x; \theta)$ as $\text{sign}(\theta \cdot x)$

Consider a classification problem:

- ▶ Distinguish between elephants ($y = 1$) and dogs ($y = 0$)
- ▶ based on some features of an animal.

Given a training set, an algorithm like the perceptron algorithm tries to find a straight line (a decision boundary) that separates the elephants and dogs.

To classify a new animal as either an elephant or a dog, it checks on which side of the decision boundary it falls, and makes its prediction accordingly.

A Different Approach

In several contexts we naturally adopt a different approach:

- ▶ Looking at elephants, build a model of what elephants look like.
- ▶ Looking at dogs, build a separate model of what dogs look like.

Finally, to classify a new animal, we can

- ▶ match the new animal against the elephant model...
- ▶ ...then match it against the dog model
- ▶ finally, see whether the new animal looks more like the elephants or more like the dogs we had seen in the training set.



Discriminative Vs. Generative

Algorithms that try to learn $p(y|x)$ directly are called **discriminative** learning algorithms.

Algorithms that instead try to model $p(x|y)$ (and $p(y)$) are called **generative** learning algorithms.

Discriminative Vs. Generative

Discriminative Approach

- ▶ model $p(y|x)$ directly
- ▶ find output that maximizes the probability for the given input

$$y = \underset{y}{\operatorname{argmax}} p(y|x)$$

Generative Approach

- ▶ model $p(x|y)$ and $p(y)$
- ▶ use Bayes' theorem

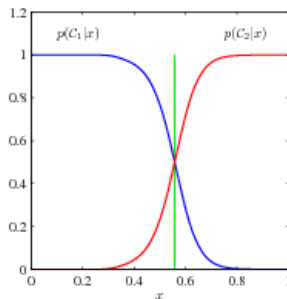
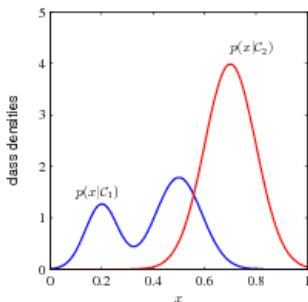
$$p(y|x) = \frac{p(x|y)p(y)}{p(x)}$$

We can avoid computing $p(x)$ since

$$\underset{y}{\operatorname{argmax}} p(y|x) = \underset{y}{\operatorname{argmax}} \frac{p(x|y)p(y)}{p(x)} = \underset{y}{\operatorname{argmax}} p(x|y)p(y)$$

Solution Complexity

Generative Vs. Discriminative



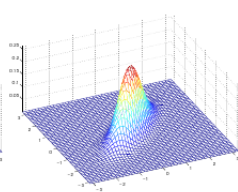
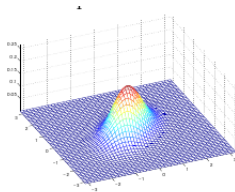
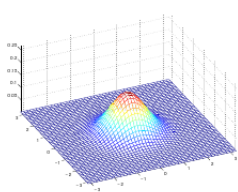
Generative

- ▶ Cons: must describe (learn) unlikely observations (complexity)
- ▶ Pros: Reject option, independent estimation (new classes/continual learning)

Example 1: Gaussian Discriminant Analysis

Assume that $p(x|y)$ is distributed according to a multivariate normal distribution.

$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right)$$



$$\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\Sigma = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}$$

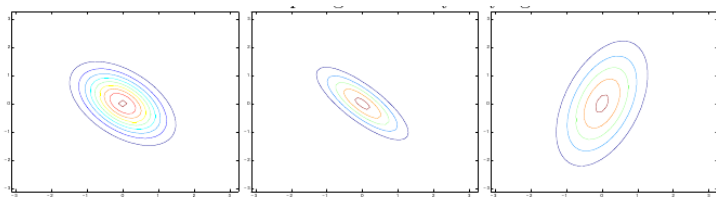
$$\Sigma = \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix}$$

Example 1: Gaussian Discriminant Analysis

The covariance Σ captures the shape of the bell

If you look at the level-sets, they are ellipses:

- ▶ direction of axis is given by **eigenvectors** of Σ
- ▶ length of axis is given by the square root of the **eigenvalues**



$$\Sigma = \begin{bmatrix} 1 & -0.5 \\ -0.5 & 1 \end{bmatrix} \quad \Sigma = \begin{bmatrix} 1 & -0.8 \\ -0.8 & 1 \end{bmatrix} \quad \Sigma = \begin{bmatrix} 3 & 0.8 \\ 0.8 & 1 \end{bmatrix}$$

Example 1: Gaussian Discriminant Analysis

Classification problem

$$\begin{aligned}y &\sim \text{Bernoulli}(\phi) \\x|y=0 &\sim \mathcal{N}(\mu_0, \Sigma) \\x|y=1 &\sim \mathcal{N}(\mu_1, \Sigma)\end{aligned}$$

Writing down the distributions explicitly

$$\begin{aligned}p(y) &= \phi^y (1 - \phi)^{1-y} \\p(x|y=0) &= \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-1/2 (x - \mu_0)^T \Sigma^{-1} (x - \mu_0) \right) \\p(x|y=1) &= \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-1/2 (x - \mu_1)^T \Sigma^{-1} (x - \mu_1) \right)\end{aligned}$$

Parameters: ϕ , μ_0 , μ_1 , Σ (single common covariance)

Maximum likelihood estimate

We estimate the parameters by maximum (log-)likelihood

- ▶ Estimated parameter is the one that maximizes (the logarithm of) the probability of observing the data

$$\begin{aligned}\ell(\phi, \mu_0, \mu_1, \Sigma) &= \log P(X, Y; \phi, \mu_0, \mu_1, \Sigma) \\ &= \log \left(\prod_{i=1}^m p(x^{(i)}, y^{(i)}; \phi, \mu_0, \mu_1, \Sigma) \right) \\ &= \sum_{i=1}^m \log \left(p(x^{(i)} | y^{(i)}; \phi, \mu_0, \mu_1, \Sigma) p(y^{(i)}; \phi) \right)\end{aligned}$$

Maximum likelihood estimate

Maximizing ℓ we obtain

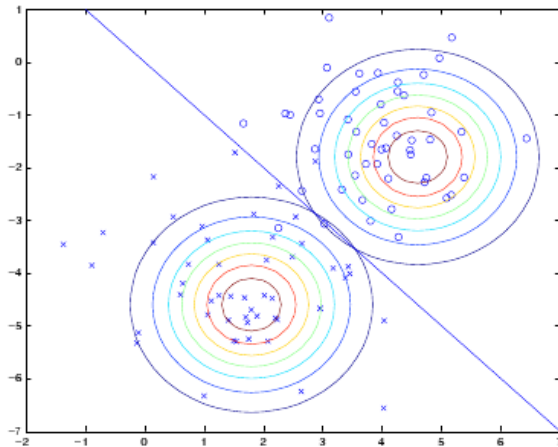
$$\phi = \frac{1}{m} \sum_{i=1}^m 1\{y^{(i)} = 1\}$$

$$\mu_0 = \frac{\sum_{i=1}^m 1\{y^{(i)} = 0\}x^{(i)}}{\sum_{i=1}^m 1\{y^{(i)} = 0\}}$$

$$\mu_1 = \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\}x^{(i)}}{\sum_{i=1}^m 1\{y^{(i)} = 1\}}$$

$$\Sigma = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu_{y^{(i)}})(x^{(i)} - \mu_{y^{(i)}})^T$$

Example 1: Gaussian Discriminant Analysis



Example 1: Naïve Bayes

Naïve Bayes is a general class of generative models, but we will introduce it in the context of an example: an email spam filter

- ▶ We wish to classify messages according to whether they are unsolicited commercial (spam) email, or non-spam (ham) email.
- ▶ Represent an email via a vector whose length is equal to the number of words in the dictionary. (**bag of words model**)
 - ▶ If an email contains the i -th word of the dictionary, then we will set $x_i = 1$; otherwise, $x_i = 0$.

$$x = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \begin{array}{l} \text{a} \\ \text{aardvark} \\ \text{aardwolf} \\ \vdots \\ \text{buy} \\ \vdots \\ \text{zygmurgy} \end{array}$$

Naïve Bayes

We have to model $p(x|y)$

But if we have, say, a vocabulary of 50000 words, then

$$x \in \{0, 1\}^{50000}$$

If we were to model x explicitly with a **multinomial distribution** over the possible outcomes, we would end up with a $(2^{50000} - 1)$ -dimensional parameter vector

Clearly too many parameters!

Naïve Bayes Assumption

Naïve Bayes assumption: assume that the x_i 's are conditionally independent given y

- ▶ If I tell you $y = 1$ (that a particular piece of email is spam), knowledge of whether “buy” appears in the message will have no effect on your beliefs about whether “price” appears
- ▶ Note that this is not the same as saying that “buy” and “price” are independent!

We now have:

$$\begin{aligned} p(x_1, \dots, x_{50000} | y) &= p(x_1 | y) p(x_2 | y, x_1) p(x_3 | y, x_1, x_2) \cdots p(x_{50000} | y, x_1, \dots, x_{49999}) \\ &= p(x_1 | y) p(x_2 | y) p(x_3 | y) \cdots p(x_{50000} | y) \\ &= \prod_{i=1}^{50000} p(x_i | y) \end{aligned}$$

Naïve Bayes

Given a training set $\{(x^{(i)}, y^{(i)}); i = 1, \dots, m\}$, we can write down the joint likelihood of the data:

$$\ell(\phi_y, \phi_{j|y=0}, \phi_{j|y=1}) = \sum_{i=1}^m \sum_{j=1}^{50000} \log p(x_j^{(i)}, y^{(i)})$$

Where $\phi_{j|y=1} = p(x_j = 1|y = 1)$, $\phi_{j|y=0} = p(x_j = 1|y = 0)$, and $\phi_y = p(y = 1)$

Naïve Bayes

Maximizing this with respect to ϕ_y , $\phi_{j|y=0}$ and $\phi_{j|y=1}$ gives the maximum likelihood estimates:

$$\begin{aligned}\phi_{j|y=1} &= \frac{\sum_{i=1}^m 1\{x_j^{(i)} = 1 \wedge y^{(i)} = 1\}}{\sum_{i=1}^m 1\{y^{(i)} = 1\}} \\ \phi_{j|y=0} &= \frac{\sum_{i=1}^m 1\{x_j^{(i)} = 1 \wedge y^{(i)} = 0\}}{\sum_{i=1}^m 1\{y^{(i)} = 0\}} \\ \phi_y &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 0\}}{m}\end{aligned}$$

Naïve Bayes

To classify as spam or ham compute

$$p(x, y = 1) = p(x|y = 1)p(y = 1) = \left(\prod_{i=1}^{50000} p(x_i|y = 1) \right) p(y = 1)$$

$$p(x, y = 0) = p(x|y = 0)p(y = 0) = \left(\prod_{i=1}^{50000} p(x_i|y = 0) \right) p(y = 0)$$

and pick whichever class has the higher joint probability.

Density Estimation

Part of the classification problem is the estimation of the *posterior class probability* $p(y|x)$

- ▶ Discriminative approaches tackle $p(y|x)$ which usually is smoother
- ▶ Discriminative classifiers learn decision boundary based on posterior: no estimate far from the boundary

Main drawback of generative model is the need to estimate the often complex $p(x|y)$ and $p(y)$

Density Estimation

We have seen parametric estimation:

- ▶ Assume that $p(x|y)$ has a specific form (e.g. is Gaussian)

Can we estimate a density without making such strong modeling assumptions?

The probability P that a vector falls in a region $\mathcal{R}$ is given by

$$P = \int_{\mathcal{R}} p(x) dx$$

If we now assume that $\mathcal{R}$ is so small that $p(x)$ does not vary much within it, we can write

$$P = \int_{\mathcal{R}} p(x) dx \approx p(x) \int_{\mathcal{R}} dx = p(x)|\mathcal{R}|$$

Density Estimation

We can make a Monte Carlo estimation of the probability as $P = \frac{k}{n}$,
i.e., the proportion of samples that fall within $\mathcal{R}$

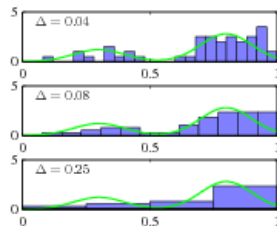
With this we have:

$$p(x) = \frac{k/n}{|\mathcal{R}|}$$

Histograms

Histograms: divide space into evenly-spaced boxes

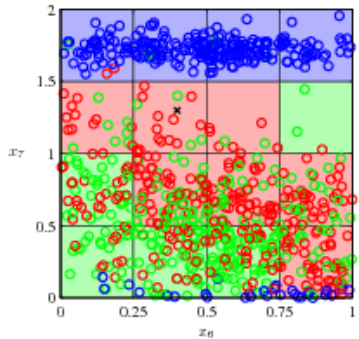
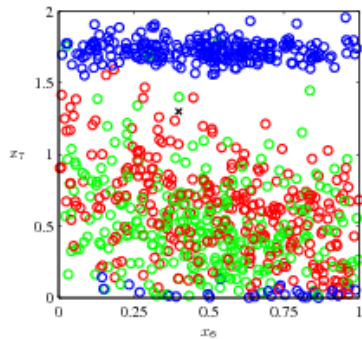
- ▶ $|\mathcal{R}|$ is constant
- ▶ regions are disjoint



Pros: Easy!

Cons: discontinuities and problems at cell boundaries

Histograms



Kernel Density Estimation (Parzen Windows)

The Parzen probability density estimation

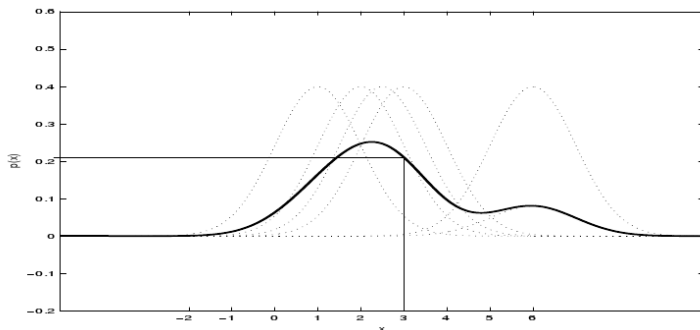
$$p(x) = \frac{1}{n} \sum_{i=1}^n \frac{1}{h^D} K \left(\frac{\|x - x_i\|}{h} \right)$$

K is a kernel or window function

- Common choice Gaussian

Kernel Density Estimation (Parzen Windows)

Intuition: average of “Bumps” of the shape of K and width h , centered on the observations x_i



Kernel Density Estimation (Parzen Windows)

How to choose h ?

Square risk is minimized by

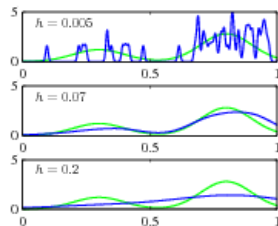
$$h^* = \frac{c_1^{-2/5} c_2^{1/5} c_3^{-1/5}}{n^{1/5}}$$

with

$$c_1 = \int x^2 K(x) dx$$

$$c_2 = \int K(x)^2 dx$$

$$c_3 = \int (f''(x))^2 dx$$





Kernel Density Estimation (Parzen Windows)

c_3 depends on underlying density f

- ▶ Solve-the-equation
- ▶ Cross-validation

No training cost (just store data)

Estimation time grows linearly with data

Window width is constant, independent of local sample density

Nearest Neighbor Estimator

Remember density estimator

$$p(x) = \frac{k/n}{|\mathcal{R}|}$$

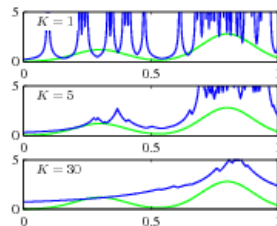
Instead of assuming constant volume $|\mathcal{R}|$ (constant width of the window)

- ▶ assume constant number of samples k
- ▶ enlarge the window until you find enough samples

The sampling window will be adaptive

- ▶ large on sparse areas
- ▶ Small on dense areas

K governs the smoothing of the data



Nearest Neighbor Estimator

Theorem

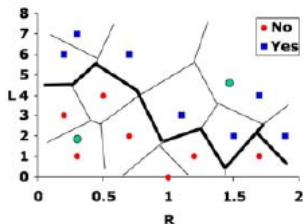
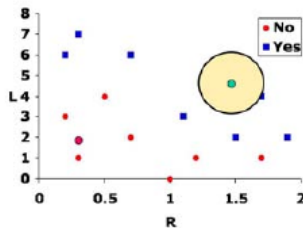
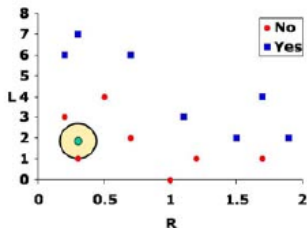
if we let the number of samples n grow to infinity, and we let the number of neighbors $k(n)$ grow to infinity more slowly than n , i.e.,

$$\lim_{n \rightarrow \infty} \frac{k(n)}{n} = 0,$$

then for each point the nearest neighbor estimator converges in probability to the real density

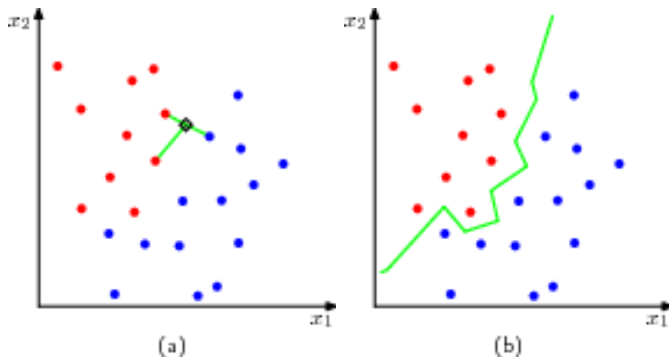
Nearest Neighbor Classifier

Nearest Neighbor rule: Look for nearest training sample and give the new point the same label



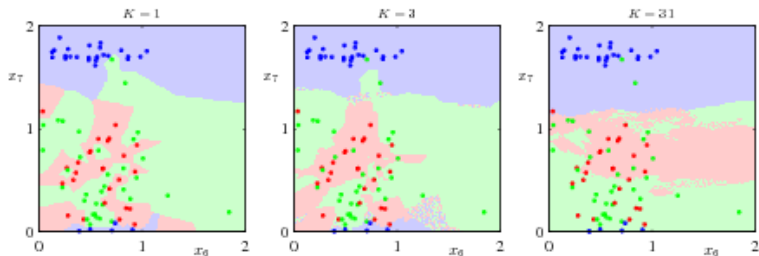
k-NN Classifier

k -NN: look for k nearest neighbor and assign label according to majority vote



k-NN Classifier

k acts as a smoothing parameter in the decision boundary



As n and k increase, k more slowly than n , k -NN converges to the optimal classifier!

k-Nearest Neighbor

Learning is fast

Lookup takes $m * n$ computations

- ▶ Clever data structures (BSP-trees) reduce to $n * \log(m)$

Big storage requirements (must keep all data)

- ▶ Delete points that are far away from the boundary (reduces lookup times as well)

Curse of dimensionality

Nearest neighbor is great in low dimensions, but as d increases things get hard

- ▶ Almost all points are far away from one another
- ▶ They are almost all near the boundary